

Informe N°2, MCDI500 PROGRAMACIÓN PARA LA CIENCIA DE DATOS

Avance Fase 3 – Semana 2 · Evaluación Formativa 3

Docente: Dr. Omar Salinas Silva

Estudiantes: Matías Manriquez Ortiz, Daniel Pérez Ramirez, Abigail Robles Chávez, Roberto Sánchez Saldivia.

Repositorio GitHub: <https://github.com/MatiasManriquezO/proyecto-grupo8-mcdi500>

Magíster en Ciencia de Datos e Inteligencia Artificial

Universidad Andrés Bello, Santiago.

Santiago, 26 de septiembre de 2026.



I. Índice

I. Índice	2
II. Introducción	3
III. Codificación funcional y arquitectura básica del script.....	4
IV. Preprocesamiento y transformación del dataset	6
V. Validación técnica y verificación del código	7
VI. Eficiencia y optimización	8
Clasificación de horas de sueño: bucle frente a vectorizado	8
Recursión: Fibonacci ingenuo frente a memoizado.....	9
VII. Diseño estructurado del código	11
VIII. Implementación modular /POO	13
IX. Documentación de arquitectura	14
X. Notebooks Ejecutables	16
XI. Repositorio GitHub.....	17
XII. Bibliografía.....	19
XIII. Anexos	20



II. Introducción

Este informe documenta el avance formativo de la Fase 3 del proyecto transversal del Grupo 8, cuya pregunta de investigación es qué relación existe entre el uso de redes sociales y la salud mental autopercebida en los estudiantes del *Youth Risk Behavior Survey 2023* (Centers for Disease Control and Prevention [CDC], 2024). En la Fase 2 construimos un pipeline funcional que deja el conjunto listo en `F2/data/processed/yrbs2023_seleccion_procesada.csv` (20.103 registros × 26 columnas). En esta entrega no incorporamos datos nuevos ni un pipeline nuevo: tomamos ese resultado, reorganizamos el código que lo procesa en funciones y clases que se pueden probar, reemplazar y reutilizar, y medimos su costo en tiempo y memoria.

El entregable central es el notebook `S2_F3_NucleoAlgoritmico_Eficiencia_POO.ipynb`, ubicado en `F3/notebooks`, ejecutado completo con numeración continua de 1 a 45 y sin errores. En él mantuvimos las dos decisiones centrales de la Fase 2: los faltantes se conservan como NA (bitácora 2.5) y las escalas ordinales no se escalan (sección 5 de F2). Los ejemplos de imputación y escalamiento de las partes 1 a 11 del notebook son didácticos y se aplican sobre una sola columna; el pipeline de las partes 12 y 13 respeta esas decisiones, y lo comprobamos con seis controles que terminan en **VERIFICACIÓN APROBADA**.

El documento sigue los apartados de la plantilla del curso. No repite el código: explica qué construimos, cómo lo verificamos, qué medimos y por qué tomamos cada decisión. Las cifras corresponden a la ejecución del notebook; los tiempos absolutos dependen del equipo, por lo que lo reproducible son las razones entre implementaciones y la forma de las curvas de crecimiento.



III. Codificación funcional y arquitectura básica del script

Organizamos el código en funciones con un solo propósito cada una y un contrato explícito de entrada y salida, siguiendo el principio de responsabilidad única del Apunte de la Fase 3. Toda la configuración (rutas, columnas y decisiones de la Fase 2) vive en una sola celda, y las rutas se arman desde la raíz del repositorio con `encontrar_raiz`, la misma convención que usamos en F1 y F2. El flujo principal queda así:

```
RAIZ = encontrar_raiz() # 1. raíz del repositorio (carpeta con .git)
datos = cargar(RUTA_DATOS) # 2. lectura y verificación del esquema
perfil = perfilar(datos) # 3. tipos, únicos y % de nulos
pipeline = construir_pipeline() # 4. pasos según la configuración
pipeline.ajustar(entrenamiento) # 5. aprende solo del entrenamiento
prueba_lista = pipeline.transformar(prueba) # 6. aplica lo aprendido a prueba
```

Tabla 1. Funciones del flujo principal

Función	Parámetros	Retorna	Responsabilidad
<code>encontrar_raiz</code>	carpeta de partida (opcional)	ruta de la raíz	Subir desde el notebook hasta encontrar <code>.git</code> ; <code>FileNotFoundError</code> si no lo encuentra
<code>leer_archivo</code>	ruta	<code>DataFrame</code>	Leer CSV, Excel o Parquet según la extensión; <code>ValueError</code> si no la reconoce
<code>verificar_esquema</code>	df, columnas esperadas	dict con declaradas y no declaradas	Comprobar que estén las 8 columnas declaradas; <code>KeyError</code> con la lista de faltantes
<code>cargar</code>	ruta, columnas esperadas	<code>DataFrame</code> 20.103 × 26	Coordinar existencia, lectura, conversión numérica y esquema
<code>perfilar</code>	df	<code>DataFrame</code> por columna	Tipo, valores únicos y porcentaje de nulos
<code>construir_pipeline</code>	imputar, escalar	Pipeline	Armar los pasos desde la configuración (patrón Factory)

Nota. Elaboración propia a partir de las celdas de configuración, carga y parte 12 del notebook.

Aplicamos las mismas convenciones en todo el notebook: nombres descriptivos en español, *docstring* en cada función y clase, comentarios solo donde la lógica no es evidente y validación de la entrada antes de trabajar. Una ruta inexistente, una extensión no reconocida o una columna faltante producen un



error con mensaje explícito, en lugar de un resultado silenciosamente incorrecto. La carga fuerza a número las cuatro escalas ordinales, porque un texto suelto las convertiría en una columna de objetos sin ningún aviso.

El flujo es verificable porque cada etapa imprime su resultado intermedio. La carga confirma 20.103 filas × 26 columnas con las 8 columnas declaradas presentes. El perfil muestra faltantes en todas las variables de análisis, entre 0,49 % (edad_cod) y 24,37 % (redes_sociales_cod), y 21,88 % en la variable objetivo salud_mental_cod (Anexo C). Con esa información decidimos cómo separar entrenamiento y prueba en el apartado siguiente.



IV. Preprocesamiento y transformación del dataset

Usamos el conjunto procesado en la Fase 2 sin repetir la limpieza. En la Fase 3 no transformamos los datos de otra forma: reorganizamos en clases las transformaciones que ya estaban decididas. Cada transformación quedó como una clase hija de Transformador con dos momentos separados: ajustar, que aprende los parámetros solo del conjunto de entrenamiento, y transformar, que los aplica a cualquier conjunto. Así evitamos la fuga de datos que tenía la primera versión, en la que la mediana se calculaba sobre el conjunto completo.

Tabla 2. Transformaciones del pipeline del proyecto

#	Transformación	Justificación técnica	Evidencia
T1	Eliminar id_registro	Identifica al estudiante, no lo describe	26 → 25 columnas
T2	One-hot de raceeth_cod	Nominal de 8 categorías: un código 6 no vale más que un 2	25 → 32 columnas; vocabulario aprendido en el ajuste
T3	One-hot de sexo_cod	Nominal de 2 categorías	32 → 33 columnas
T4	Separar entrenamiento y prueba	Solo registros con salud_mental_cod respondida: la variable objetivo no se imputa	15.705 de 20.103: 12.564 y 3.141 (80/20, semilla 42)

Nota. Elaboración propia a partir de las partes 4, 12 y 13 del notebook.

Declaramos también lo que **no** hicimos, porque es parte de la decisión. No imputamos ni escalamos las escalas ordinales: el pipeline se construye desde la configuración con IMPUTAR_ORDINALES = False y ESCALAR_ORDINALES = False, por lo que conserva 5.687 NA en las cuatro escalas ordinales de los 15.705 registros y deja los códigos del codebook tal como vienen. Tampoco ponderamos: peso_muestral, estrato y psu se conservan intactos para el análisis ponderado de la Fase 4.

Para respaldar la decisión de no imputar con evidencia, comparamos tres estrategias sobre actividad_fisica_cod (6,10 % de faltantes) con el patrón Strategy. La mediana y la media reducen la desviación estándar de 2,503 a 2,425 (-3,10 %), y la mediana por sexo, a 2,432 (-2,84 %). Toda imputación por un valor central reduce la dispersión; como en la Fase 2 mostramos que los faltantes no son aleatorios, mantenemos los NA y dejamos la imputación solo como ejemplo didáctico.



V. Validación técnica y verificación del código

Organizamos la verificación en tres escenarios —caso normal, casos límite y excepciones— con sentencias `assert`, que detienen la ejecución en el punto donde se origina un error. El notebook se ejecutó completo con *Restart Kernel and Run All*, sin errores y con numeración continua de 1 a 45.

Tabla 3. Evidencia de verificación

Escenario	Qué comprobamos	Resultado
Normal	Pipeline sobre las 20.103 filas: no pierde filas, elimina el identificador y no deja nulos en la columna imputada	3 <code>assert</code> superados
Límite	Columna sin nulos · una sola categoría · varianza cero · categoría nueva que aparece solo en prueba	Sin cambios · 1 columna · 0,0 sin división por cero · no crea columna
Excepción	Transformar sin ajustar · columna inexistente · agregar algo que no es Transformador · pipeline sin ajustar	<code>RuntimeError</code> , <code>KeyError</code> , <code>TypeError</code> y <code>RuntimeError</code> , con mensaje claro
Encapsulamiento	Transformar antes de ajustar · asignar la propiedad ajustado · usar la clase base sola	<code>RuntimeError</code> , <code>AttributeError</code> y <code>NotImplementedError</code>
Contra la Fase 2	Columnas one-hot de CodificadorNominal frente a las 8 columnas <code>raza_*</code> guardadas por F2	Coinciden en 19.733 filas; las 370 sin raza siguen como NA
Resultado final	Filas, esquema, NA conservados, escalas sin escalar, objetivo completo e identificador eliminado	6 controles OK: VERIFICACIÓN APROBADA (Anexo B)
Persistencia	Relectura de las salidas guardadas en F3/data/_demo	Misma forma (12.564 × 33), mismas columnas y mismos NA

Nota. Elaboración propia a partir de las partes 4, 8 y 13 del notebook.

La comprobación de mayor valor es la comparación con la Fase 2: el resultado de la clase se contrasta con el archivo que produjo F2, que es un camino independiente. Si dos procedimientos distintos coinciden, un error de lógica en uno de ellos es improbable. También verificamos la ausencia de fuga de datos: la media de `edad_cod` escalada en prueba es $-0,013$ y no cero, porque los parámetros vienen solo del entrenamiento. Por último, antes de cada medición de tiempo comprobamos con `assert` que las implementaciones comparadas entregan el mismo resultado, porque una versión más rápida que entrega otro resultado no es una optimización, sino un error.



VI. Eficiencia y optimización

Seguimos el protocolo de medición de la Semana 2: repetir cada medición, medir en el tamaño real y en tamaños crecientes, y comprobar antes la equivalencia de resultados. El tiempo se midió con `timeit` y `perf_counter`, y la memoria pico con `tracemalloc`, todos de la biblioteca estándar (Python Software Foundation, s. f.-a, s. f.-b). Encapsulamos la medición en la función `medir`, que recibe cualquier función con sus argumentos y devuelve resultado, segundos y memoria pico.

Clasificación de horas de sueño: bucle frente a vectorizado

Clasificamos `sueno_cod` en tres categorías (muy insuficiente, hasta 5 horas; insuficiente, 6 a 7 horas; suficiente, 8 horas o más) de dos formas: un bucle que recorre las 20.103 filas una por una y una versión vectorizada con `pd.cut`. Ambas son $O(n)$ en teoría; lo que se mide es la constante por fila, donde está la diferencia práctica (Cormen et al., 2022).

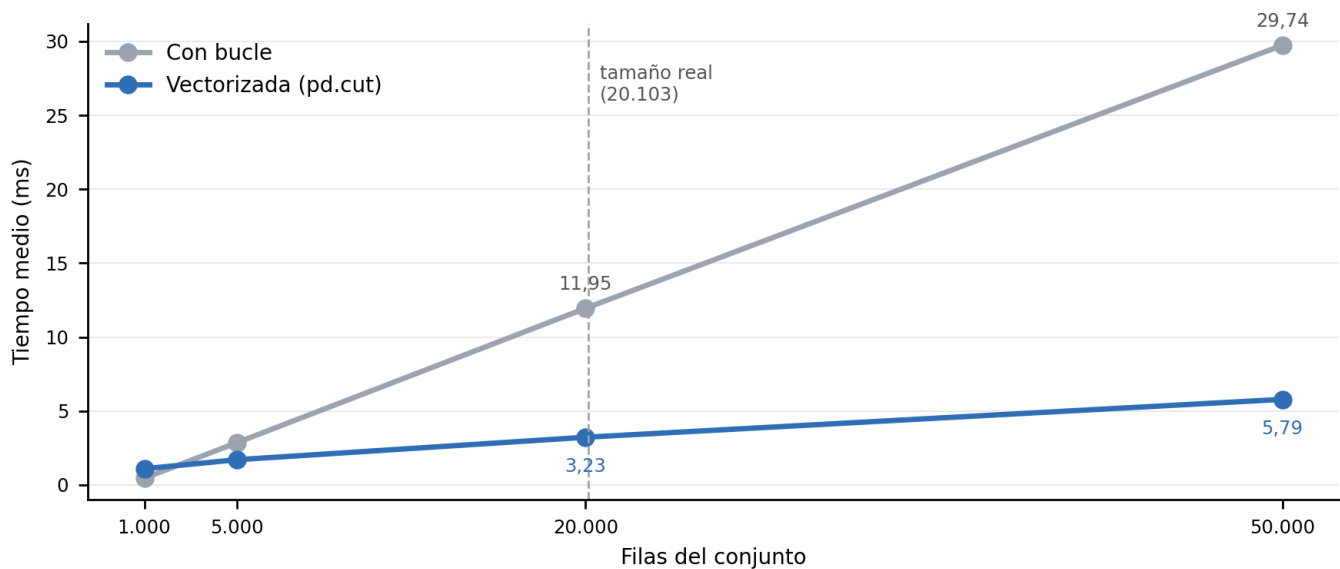
Tabla 4. Clasificación de sueño en el tamaño real (n = 20.103)

Implementación	20 repeticiones (s)	Una ejecución (ms)	Memoria pico (MB)
Con bucle	0,2205	14,21	0,165
Vectorizada (<code>pd.cut</code>)	0,0500	8,09	0,513

Nota. La columna de una ejecución se midió junto con `tracemalloc`, que agrega costo a ambas versiones. Los tiempos dependen del equipo; lo reproducible es la razón entre versiones (4,4 veces en 20 repeticiones).



Figura 1. Tiempo medio de la clasificación de sueño según tamaño del conjunto



Nota. Promedio de 5 repeticiones por tamaño; la línea discontinua marca el tamaño real. Los valores se encuentran en el Anexo A.

La lectura tiene tres niveles. Primero, el bucle crece en proporción directa al tamaño: pasa de 0,46 ms con 1.000 filas a 29,74 ms con 50.000, característico de un algoritmo lineal. Segundo, la versión vectorizada tiene un costo fijo de preparación que domina en tamaños pequeños: con 1.000 filas es más de dos veces más lenta que el bucle (razón 0,4). Tercero, la ventaja aparece desde 5.000 filas y crece con el volumen: 1,7 veces con 5.000, 3,7 con 20.000 y 5,1 con 50.000. A cambio, la vectorizada consume unas tres veces más memoria (0,51 frente a 0,17 MB), porque construye una columna categórica intermedia.

Decisión. Adoptamos la versión vectorizada para el tamaño real del proyecto y superiores; su costo en memoria es menor a 1 MB y aceptable. La medición nos corrigió una idea previa: vectorizar no siempre gana, y por eso reportamos en qué condiciones conviene cada versión.

Recursión: Fibonacci ingenuo frente a memoizado

La segunda medición contrasta dos funciones recursivas con la misma idea. La versión ingenua recalcula los mismos subproblemas muchas veces; la memoizada guarda en un diccionario lo ya calculado.



Tabla 5. Fibonacci recursivo: ingenuo frente a memoizado

n	Ingenua (ms)	Memoizada (ms)	Veces más rápida
18	0,708	0,020	35,9
22	5,223	0,027	193,5
26	46,047	0,042	1.088,6

Nota. Una ejecución por valor de n, medida con perf_counter.

- VII. La versión ingenua crece de forma exponencial: cada vez que n aumenta en 4, el tiempo se multiplica entre 7 y 9 veces. La memoizada crece de forma lineal y se mantiene bajo 0,05 ms. El cambio entre ambas es de dos líneas, y el efecto se mide en cientos o miles de veces. Lo que hace eficiente a la segunda no es la recursión, sino no repetir trabajo; el costo es memoria, porque el diccionario guarda n resultados. Es el mismo intercambio entre tiempo y memoria de la medición anterior.

VIII. Diseño estructurado del código

El diseño combina programación estructurada —funciones y clases con estructuras de control y una sola responsabilidad— con recursividad solo donde el problema la justifica. Nuestro criterio, tomado del Apunte de la Fase 3, es que la recursión conviene cuando la profundidad del problema no se conoce de antemano; si la profundidad es fija o el recorrido es secuencial, un bucle es más simple y rápido.

Tabla 6. Casos de recursión evaluados en el notebook

Caso	Profundidad	Resultado	Decisión
aplanar metadatos anidados	Desconocida	Convierte un diccionario de hasta 3 niveles en claves planas sin cambiar el código	Adoptada
cuenta_regresiva	n	Muestra el caso base y el caso recursivo	Ejemplo didáctico
Fibonacci ingenua	Llamadas exponenciales	46,05 ms con $n = 26$	Descartada
Fibonacci memoizada	n	1.089 veces más rápida que la ingenua	Ejemplo de memoización
Pasos del pipeline	Fija	Se recorren con un bucle en Pipeline	Bucle

Nota. Elaboración propia a partir de la parte 9 del notebook.

Cohesión y acoplamiento. Contrastamos una clase de cohesión baja, `ProcesadorTodoEnUno`, que carga, limpia, gráfica y envía correos, con dos clases de cohesión alta, `Cargador` y `Limpiador`. La ventaja se vio al probar: el `Limpiador` se validó sin tocar el disco (3 filas de entrada, 2 de salida). Para medir el acoplamiento usamos una pregunta concreta: si cambiamos la forma de imputar, ¿cuántos archivos hay que modificar? Cambiamos el paso de imputación de `actividad_fisica_cod` a `sueno_cod` y el Pipeline no se tocó; ambas versiones entregan 20.103×25 .

Escalabilidad. El diseño está preparado para crecer sin reescribirse. Un paso nuevo se agrega como una clase hija de `Transformador` que implementa aprender y aplicar; el pipeline, la validación y la bitácora del patrón `Observer` lo incorporan sin cambios. Como la configuración gobierna el pipeline, cambiar una columna o una decisión se hace en una sola celda. En la Fase 4, el análisis ponderado con `peso_muestral`, `estrato` y `psu` podrá agregarse como un paso más. Esta proyección se explica, pero no se implementa en



este avance.



IX. Implementación modular /POO

La guía de la Formativa 3 deja las clases como opcionales en esta entrega. Las incorporamos porque el pipeline necesita recordar lo que aprende entre ajustar y transformar, que es nuestro criterio para decidir entre una función y una clase: si el componente debe recordar algo entre llamadas, es una clase; si no, basta una función. La parte 2 del notebook deja por escrito los conceptos base que usamos (clase, objeto, atributo, método y self).

Tabla 7. Principios de la POO aplicados y su evidencia

Principio	Cómo lo aplicamos	Evidencia en el notebook
Encapsulamiento	<code>_parametros</code> y <code>_ajustado</code> internos; propiedades nombre y parametros de solo lectura, con copia defensiva	Transformar sin ajustar lanza <code>RuntimeError</code> ; asignar ajustado lanza <code>AttributeError</code>
Herencia	<code>ImputadorMediana</code> , <code>CodificadorNominal</code> , <code>EscaladorEstandar</code> , <code>EliminadorColumnas</code> e <code>ImputadorFlexible</code> heredan de <code>Transformador</code>	MRO: <code>ImputadorMediana</code> → <code>Transformador</code> → <code>object</code> ; <code>ajustar</code> , <code>transformar</code> y <code>ajustar_transformar</code> se heredan
Polimorfismo	Pipeline recorre los pasos llamando a <code>ajustar_transformar</code> sin preguntar su tipo	5 pasos distintos en un solo bucle: 20.103×33

Nota. Elaboración propia a partir de las partes 3 a 6 del notebook.

Revisamos cuatro patrones de diseño y adoptamos **Strategy** como principal, aplicado al tratamiento de faltantes: `ImputadorFlexible` sabe cuándo imputar y la estrategia (mediana, media o mediana por grupo) aporta el cómo. **Factory** aparece en `crear_transformador` y `construir_pipeline`, que eligen la transformación según el rol de la variable. **Observer** se aplica en `PipelineObservable`, que avisa a `Bitacora` y `ReporteConsola` cada vez que termina un paso, de modo que la bitácora de ejecución se genera sola. **Singleton**, en la clase `Configuracion`, lo mostramos como referencia y lo limitamos a la configuración, porque introduce estado global.

Organización modular inicial. En este avance las clases viven en el notebook, agrupadas por responsabilidad. La parte 13 propone la separación en archivos que corresponde llevar al repositorio: `transformadores.py` para la clase base y sus hijas, `pipeline.py` para `Pipeline` y `PipelineObservable`, `observadores.py`, `fabrica.py` y `medicion.py`. Esa estructura se resume en la Tabla 8.



X. Documentación de arquitectura

La tabla de componentes la generamos en el notebook (parte 13.2) leyendo las clases que existen con `__subclasses__()` y sus *docstrings*, de modo que refleja el código real y no puede quedar desactualizada.

Tabla 8. Componentes, responsabilidades y ubicación propuesta

Capa	Componente	Responsabilidad	Archivo propuesto
Configuración	encontrar_raiz, constantes de la celda de configuración	Rutas desde la raíz, columnas y decisiones de F2	notebook
Entrada	leer_archivo, verificar_esquema, cargar	Leer el archivo de F2 y verificar su esquema	src/carga.py
Núcleo	Transformador	Define el contrato y controla el estado de cada paso	src/transformadores.py
Núcleo	ImputadorMediana, CodificadorNominal, EscaladorEstandar, EliminatorColumnas, ImputadorFlexible	Cada una, un paso del pipeline	src/transformadores.py
Orquestación	Pipeline, PipelineObservable	Encadenar los pasos y controlar el orden de ajuste	src/pipeline.py
Orquestación	construir_pipeline	Armar el pipeline desde la configuración	src/fabrica.py
Soporte	Bitacora, ReporteConsola	Registrar lo ocurrido en cada paso	src/observadores.py
Soporte	medir	Medir tiempo y memoria de cualquier función	src/medicion.py

Nota. Elaboración propia a partir de la parte 13.2 del notebook.

Las dependencias van en un solo sentido: la configuración alimenta la carga, la carga entrega el conjunto al pipeline y el pipeline no conoce el detalle de cada paso. El flujo es el siguiente:

```
configuración (RAIZ, rutas, columnas, decisiones de F2)
└─> cargar() → perfilar()
    └─> construir_pipeline() → Pipeline
        ├── EliminatorColumnas(id_registro)
        ├── CodificadorNominal(raceeth_cod)
        └── CodificadorNominal(sexo_cod)
└─> verificar_resultado() → guardar_salidas_demo() → F3/data/_demo/
```

Justificación. Organizamos el código de este modo por tres razones. Primero, la clase base



concentra el control de estado, de modo que cada paso nuevo solo implementa lo que cambia; si hay que ajustar un mensaje de error, se cambia en un solo lugar. Segundo, el Pipeline no conoce el detalle de los pasos: esa independencia (bajo acoplamiento) permite cambiar la estrategia de imputación sin tocar el orquestador. Tercero, cada componente tiene un propósito que se enuncia en una frase, que es el criterio de alta cohesión del Apunte de la Fase 3.

Evolución. En la evaluación sumativa proyectamos: (a) mover las clases del notebook a los archivos de F3/src/ propuestos en la Tabla 8; (b) convertir cada paso real de la Fase 2 en una clase hija y comprobar que el pipeline de clases reproduce el archivo de F2; y (c) agregar en la Fase 4 un paso de ponderación con el diseño muestral. **Vinculación:** la bitácora (docs/bitacora_decisiones.md) registra las decisiones de la Fase 2 que este notebook respeta, y el README describe el orden de ejecución de los notebooks.



XI. Notebooks Ejecutables

El entregable central es F3/notebooks/S2_F3_NucleoAlgoritmico_Eficiencia_POO.ipynb, con 110 celdas, de las cuales 45 son de código. Se ejecutó completo con *Restart Kernel and Run All Cells*, sin errores, con numeración continua de 1 a 45 y todas las salidas visibles. Cada bloque de código está precedido por una celda de texto que explica qué hace y seguido, cuando corresponde, por su interpretación, práctica recomendada para análisis reproducibles en Jupyter (Rule et al., 2019).

La reproducibilidad descansa en cuatro elementos: rutas relativas a la raíz del repositorio con encontrar_raiz (la misma convención de F1 y F2), semilla fija (42), dependencias declaradas en el requirements.txt de la raíz y salidas persistidas con verificación de ida y vuelta. Las salidas se guardan en F3/data/_demo/ con el prefijo demo_ y el notebook aclara que no reemplazan el dataset oficial de la Fase 2.

Tabla 9. Estructura del notebook

Parte	Contenido	Apartado del informe
—	Configuración, carga y perfil del conjunto	III
1–2	De funciones a objetos; conceptos base de la POO	III y VIII
3–6	Clase Preprocesador, encapsulamiento, herencia, polimorfismo y Pipeline	IV y VIII
7	Cohesión y acoplamiento	VII y IX
8	Validación: normal, límite, excepción y comparación con F2	V
9	Recursividad	VI y VII
10	Eficiencia: tiempo, memoria y crecimiento	VI
11	Patrones de diseño: Strategy, Factory, Observer y Singleton	VIII
12–13	Pipeline desde la configuración, arquitectura, verificación y salidas	IV, V y IX

Nota. Elaboración propia.

Como limitación, los tiempos absolutos dependen del equipo en que se ejecute el notebook; lo reproducible son las razones entre implementaciones y la forma de las curvas de crecimiento.

XII. Repositorio GitHub

El repositorio del proyecto (<https://github.com/MatiasManriquezO/proyecto-grupo8-mcdi500>) conserva la estructura por fases: cada fase guarda sus propios notebooks, datos y documentación, y docs/ reúne lo que cruza todo el proyecto. Las carpetas F1 y F2 se mantienen sin cambios, como referencia; la salida de F2 es la entrada de F3. La estructura completa, con lo incorporado en la Fase 3, es la siguiente:

```
proyecto-grupo8-mcdi500/
├── src/
│   └── procesamiento.py           funciones compartidas por las fases (F2, sin cambios)
├── F1/
│   ├── data/raw/XXH2023_YRBSS_data.csv  dataset original YRBS 2023 (CDC)
│   ├── data/docs/                      diccionario, criterios, metadatos, vinculación
│   └── notebooks/S1_F1_Definicion.ipynb
├── F2/
│   ├── data/processed/yrbs2023_seleccion_procesada.csv  conjunto oficial (entrada de F3)
│   ├── docs/                      diccionario, metadatos y registro de F2
│   └── notebooks/S1_F2_Preprocesamiento.ipynb
├── F3/
│   ├── notebooks/
│   │   ├── S2_F3_NucleoAlgoritmico_Eficiencia_POO.ipynb  este avance (Formativa 3)
│   │   └── S2_F3_NucleoAlgoritmico_POO_Grupo8.ipynb      pipeline real (Sumativa 2)
│   ├── src/
│   │   └── núcleo algorítmico: contingencia.py, busqueda.py,
│   │       recursion.py, medicion.py, analisis.py
│   ├── data/_demo/
│   │   └── salidas de este notebook (prefijo demo_)
│   │   ├── demo_entrenamiento.csv · demo_prueba.csv
│   │   ├── demo_diccionario.csv · demo_parametros.csv
│   │   └── demo_entrenamiento_config.csv · demo_entrenamiento_config_parametros.csv
│   ├── resultados/
│   │   └── tablas de mediciones y del pipeline de clases
│   │   ├── tabla4...tabla7, tabla_proporciones.csv, arquitectura_codigo.csv  (F3/src)
│   │   └── parametros_pipeline_f3.csv, bitacora_ejecucion_f3.csv, arquitectura_f3.csv,
│   │       eficiencia_faltantes_f3.csv, eficiencia_crecimiento_f3.csv          (Sumativa 2)
├── F4/notebooks/Fase 4.md           pendiente
├── docs/
│   ├── bitacora_decisiones.md        decisiones F1-F3 (F3.1 a F3.18) e infraestructura
│   ├── diccionario_variables.md
│   └── Informe/                      informes entregados (PDF)
│       └── Mapa Conceptual Proyecto/
├── .gitignore · .mailmap
├── requirements.txt                  dependencias (único, en la raíz)
└── README.md                        estructura, ejecución y decisiones
```

En esta entrega actualizamos el README con la estructura completa, el orden de ejecución de los cuatro notebooks y la descripción de cada uno; no se requieren librerías nuevas, porque timeit, tracemalloc



e inspect pertenecen a la biblioteca estándar. La bitácora incorpora las decisiones F3.11 a F3.18 y las de infraestructura 3.8 y 3.9. Eliminamos F3/data/processed/, que contenía salidas de una versión anterior del notebook que imputaba y escalaba, en contra de las decisiones 2.4 y 2.5; en su lugar quedó F3/data/_demo/.

Los cambios se incorporan con la convención de *commits* del proyecto, sin tildes en los mensajes: feat: actualiza notebook F3 formativa con clases, eficiencia y patrones, data: agrega salidas de demostracion en F3/data/_demo, fix: elimina F3/data/processed con salidas imputadas y escaladas y docs: actualiza README y bitacora con decisiones F3.11 a F3.18. La autoría queda trazable por integrante con git shortlog -sne, que el archivo .mailmap unifica cuando una misma persona commiteó con más de un nombre.



XIII. Bibliografía

- Centers for Disease Control and Prevention. (2024). *2023 Youth Risk Behavior Survey data* [Conjunto de datos]. <https://www.cdc.gov/yrbs/data/index.html>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). MIT Press.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- McKinney, W. (2022). *Python for data analysis* (3.^a ed.). O'Reilly Media.
- Python Software Foundation. (s. f.-a). *timeit — Measure execution time of small code snippets*. <https://docs.python.org/3/library/timeit.html>
- Python Software Foundation. (s. f.-b). *tracemalloc — Trace memory allocations*. <https://docs.python.org/3/library/tracemalloc.html>
- Rule, A., Birmingham, A., Zuniga, C., Altintas, I., Huang, S.-C., Knight, R., Moshiri, N., Nguyen, M. H., Rosenthal, S. B., Pérez, F., & Rose, P. W. (2019). Ten simple rules for writing and sharing computational analyses in Jupyter Notebooks. *PLOS Computational Biology*, 15(7), e1007007. <https://doi.org/10.1371/journal.pcbi.1007007>
- Universidad Andrés Bello. (2026a). *Apunte de la Fase 3: Núcleo algorítmico, eficiencia y programación orientada a objetos* [Material docente]. MCDI500 Programación para la Ciencia de Datos.
- Universidad Andrés Bello. (2026b). *Guía de apoyo: Evaluación Formativa 3 (Fase 3). Notebook con scripts y mediciones de complejidad algorítmica* [Material docente]. MCDI500 Programación para la Ciencia de Datos.

XIV. Anexos

Anexo A. Clasificación de sueño: tiempo medio (ms) según tamaño

n (filas)	Con bucle	Vectorizada	Bucle / vectorizada
1.000	0,46	1,10	0,4
5.000	2,87	1,71	1,7
20.000	11,95	3,23	3,7
50.000	29,74	5,79	5,1

Nota. Muestras con reemplazo del conjunto de la Fase 2 (semilla 42), solo para medir. Promedio de 5 repeticiones por tamaño.

Anexo B. Verificación del resultado del pipeline

Control	Estado	Detalle
Se conservan todas las filas	OK	15.705 de 15.705
Mismo esquema en entrenamiento y prueba	OK	33 columnas
Faltantes conservados como NA	OK	5.687 NA
Escalas ordinales sin escalar	OK	4 columnas
Variable objetivo presente y completa	OK	salud_mental_cod
Identificador eliminado	OK	id_registro

Nota. Salida de la función verificar_resultado (parte 13.4 del notebook).

Anexo C. Faltantes en las variables de análisis (n = 20.103)

Variable	Valores distintos	Nulos	% de nulos
redes_sociales_cod	8	4.900	24,37
salud_mental_cod	5	4.398	21,88
sueno_cod	7	2.662	13,24
actividad_fisica_cod	8	1.227	6,10
raceeth_cod	8	370	1,84
sexo_cod	2	158	0,79
edad_cod	7	98	0,49

Nota. Salida de la función perfilar sobre el archivo de la Fase 2. Elaboración propia con datos del YRBS 2023 (CDC, 2024).